

```

Widget& operator=(const Widget& rhs)           //返回类型是个 reference,
{                                              // 指向当前对象。
    ...
    return* this;                             //返回左侧对象
}
...
};

```

这个协议不仅适用于以上的标准赋值形式，也适用于所有赋值相关运算，例如：

```

class Widget {
public:
    ...
    Widget& operator+=(const Widget& rhs)      //这个协议适用于
    {                                          // +=, -=, *=, 等等。
        ...
        return *this;
    }
    Widget& operator=(int rhs)                //此函数也适用，即使
    {                                          // 此一操作符的参数类型
        ...                                  // 不符协定。
        return *this;
    }
    ...
};

```

注意，这只是个协议，并无强制性。如果不遵循它，代码一样可通过编译。然而这份协议被所有内置类型和标准程序库提供的类型如 `string`, `vector`, `complex`, `tr1::shared_ptr` 或即将提供的类型（见条款 54）共同遵守。因此除非你有一个标新立异的好理由，不然还是随众吧。

请记住

- 令赋值（*assignment*）操作符返回一个 `reference to *this`。

条款 11: 在 operator= 中处理 “自我赋值”

Handle assignment to self in operator=.

“自我赋值”发生在对象被赋值给自己时：

```

class Widget { ... };
Widget w;
...
w = w;                                     //赋值给自己

```

这看起来有点愚蠢，但它合法，所以不要认定客户绝不会那么做。此外赋值动

作并不总是那么可被一眼辨识出来，例如：

```
a[i] = a[j];           //潜在的自我赋值
```

如果 *i* 和 *j* 有相同的值，这便是个自我赋值。再看：

```
*px = *py;             //潜在的自我赋值
```

如果 *px* 和 *py* 恰巧指向同一个东西，这也是自我赋值。这些并不明显的自我赋值，是“别名”（*aliasing*）带来的结果：所谓“别名”就是“有一个以上的方法指称（指涉）某对象”。一般而言如果某段代码操作 *pointers* 或 *references* 而它们被用来“指向多个相同类型的对象”，就需考虑这些对象是否为同一个。实际上两个对象只要来自同一个继承体系，它们甚至不需声明为相同类型就可能造成“别名”，因为一个 *base class* 的 *reference* 或 *pointer* 可以指向一个 *derived class* 对象：

```
class Base { ... };
class Derived: public Base { ... };
void doSomething(const Base& rb,           //rb 和 *pd 有可能其实是同一对象
                 Derived* pd);
```

如果遵循条款 13 和条款 14 的忠告，你会运用对象来管理资源，而且你可以确定所谓“资源管理对象”在 *copy* 发生时有着正确的举措。这种情况下你的赋值操作符或许是“自我赋值安全的”（*self-assignment-safe*），不需要额外操心。然而如果你尝试自行管理资源（如果你打算写一个用于资源管理的 *class* 就得这样做），可能会掉进“在停止使用资源之前意外释放了它”的陷阱。假设你建立一个 *class* 用来保存一个指针指向一块动态分配的位图（*bitmap*）：

```
class Bitmap { ... };
class Widget {
    ...
private:
    Bitmap* pb;           //指针，指向一个从 heap 分配而得的对象
};
```

下面是 *operator=* 实现代码，表面上看起来合理，但自我赋值出现时并不安全（它也不具备异常安全性，但我们稍后才讨论这个主题）。

```
Widget&
Widget::operator=(const Widget& rhs) //一份不安全的 operator= 实现版本.
{
    delete pb;               //停止使用当前的 bitmap,
    pb = new Bitmap(*rhs.pb); //使用 rhs's bitmap 的副本（复件）。
    return *this;            //见条款 10.
}
```